🎮 Joystick

Hardware Overview

🎮 What is it?

The **Joystick** (Model KY-023) is your main input device for movement. On the K-Comp, you can use it to control a character in a game, steer a robot, or navigate menus.

⚡ Electrical Explanation

Inside the joystick, there are **two potentiometers** (variable resistors) at a 90° angle to each other.

- One measures the **Left/Right** movement.
- One measures the **Up/Down** movement.

As you move the stick, it turns the internal wipers, changing the resistance. This changes the output voltage between 0V and 5V. The Processor reads this voltage and converts it into a number.

- **Connection (Y-Axis):** Connected to **Analog Pin A1**.
- **Connection (X-Axis):** Connected to **Analog Pin A2**.
- **Signal Flow:** Movement ➔ Resistance Change ➔ Voltage Change (0-5V) ➔ ADC (Analog to Digital Converter) ➔ Digital Number.

📉 How Coordinates Work: The "Centered" System

Standard analog inputs read a value from 0 to 1023. However, the **K-Comp library** automatically does the math to make the **center position 0**.

- **Negative Numbers:** left or down.
- **Positive Numbers:** right or up.
- **Zero:** center (resting position).

Stick Position	Raw Value	K-Comp Library Value
Left	0	-512
Center	~512	0
Right	1023	+511

Software & Programming

📚 kcomp_joystick.h

This library processes the raw data for you. It can calculate the **Direction** (e.g., "UP_LEFT") and the **Intensity** (0-100%) automatically. **Usage:** `#include <kcomp_joystick.h>`.

Function Reference

Function Name	Description & Parameters
`void initJoystick()`	**Description:** Initializes the joystick. Sets pins A1 and A2 as inputs. **Usage:** Must be called once inside `setup()`.
`int getJoystickXValue()`	**Description:** Gets the position of the X-axis (horizontal). **Returns:** A value from -512 (left) and +511 (right).
`int getJoystickYValue()`	**Description:** Gets the position of the Y-axis (vertical). **Returns:** A value from -512 (Down) and +511 (Up).
`joystickState getJoystickState()`	**Read Everything:** The "All-in-One" function. **Returns:** A `joystickState` structure containing both `.direction` and `.intensity` calculated for you.

🧭 Joystick Directions (States)

When you access `state.direction`, it will be exactly **one** of the values:

`JOYSTICK_CENTER`, `JOYSTICK_UP`, `JOYSTICK_DOWN`, `JOYSTICK_LEFT`, `JOYSTICK_RIGHT`, `JOYSTICK_UP_LEFT`, `JOYSTICK_UP_RIGHT`, `JOYSTICK_DOWN_LEFT`, `JOYSTICK_DOWN_RIGHT`, `JOYSTICK_UNDEFINED`

How to use it?

```
if (state.direction == JOYSTICK_UP_RIGHT)
{
    // Move character diagonally
}
```

⚠️ Pitfall: The "Deadzone" (Gap)

Mechanical joysticks are never perfect; when you let go, they might not settle at exactly `0` (it might be `2` or `−5`). Hence, the library ignores any movement smaller than `50`.

📝 Sample Program

This program prints the direction and intensity to the Serial Monitor.

```
#include <kcomp_joystick.h>

void setup() {
    Serial.begin(115200);
    initJoystick();       // Initialize the hardware
    Serial.println("Joystick Ready!");
}

void loop() {
    // Get the calculated state (Direction + Intensity)
    joystickState state = getJoystickState(); //

    // Print Intensity (0% to 100%)
    Serial.print("Speed: ");
    Serial.print(state.intensity);
    Serial.print("%");

    // Check specific directions
    if (state.direction == JOYSTICK_UP) {
        Serial.println(" -> Moving UP");
    }
    else if (state.direction == JOYSTICK_CENTER) {
        Serial.println(" -> Stopped");
    }

    delay(200);
}
```

💡 **Try it out:** You can find the full sample program **SimpleJoystick** in the K-Comp examples folder!